

Chocolate Doom Telemetry & UX Database

Entrega 3 - Consultas, Indexacion y Reporte

Proyecto de Bases de Datos - DBS Semester Project

Integrantes: Juan Sebastian Ruiz y German David Rodriguez

Este informe consolida las entregas 1, 2 y 3. Incluye diseno conceptual y logico, implementacion en PostgreSQL, ingesta ETL de telemetria real de seis jugadores, consultas analiticas, indices, vistas, vista materializada y mecanismo de reproduccion.

1. Alcance y estado de la entrega

El proyecto modela una base de datos relacional para almacenar telemetria generada por Chocolate Doom y relacionarla con informacion anonima de usuarios y resultados del instrumento UX PENS. La entrega actual usa una base unificada con seis jugadores reales.

Metrica	Valor
Filas de telemetria reales	5686
Jugadores registrados	6
Jugadores con telemetria real	6
Sesiones reales	50
Primera muestra	2026-06-02 19:30:14
Ultima muestra	2026-06-04 21:37:00

Resumen por jugador:

Jugador	Eventos	Sesiones	Primera muestra	Ultima muestra	Mapas	Episodios
player_01	2326	26	2026-06-02 19:30:14	2026-06-03 23:02:12	3	4
player_02	509	2	2026-06-04 18:39:36	2026-06-04 18:54:51	2	2
player_03	492	4	2026-06-04 19:02:40	2026-06-04 20:30:09	3	2
player_04	292	2	2026-06-04 20:37:12	2026-06-04 20:43:02	2	2
player_05	694	6	2026-06-04 20:48:22	2026-06-04 21:37:00	3	3
player_06	1373	10	2026-06-04 18:32:04	2026-06-04 19:08:25	4	4

2. Parte A - Diseno conceptual y logico

2.1 Supuestos principales

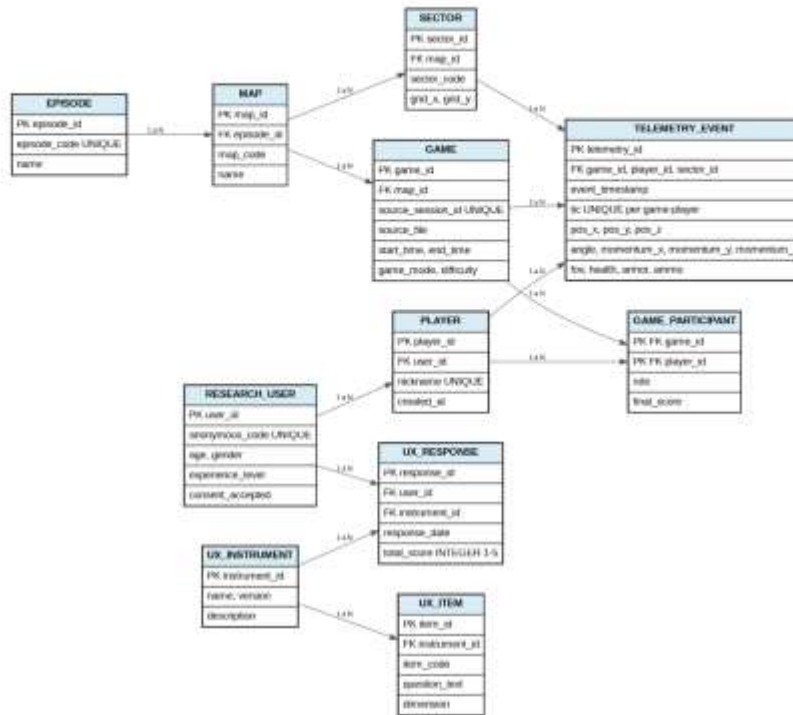
Supuesto	Descripcion
Usuarios anonimos	Los participantes se identifican mediante anonymous_code y no se almacenan nombres reales.
Relacion User-Player	Un usuario puede tener uno o mas alias de juego.
Sesion de juego	Cada bloque When del log se representa como una sesion GAME con source_session_id.
Telemetria por tic	Cada fila de captura se representa como un TELEMETRY_EVENT.
Sectores	El sector se modela como una celda de grilla asociada a un mapa.
UX	Se usa PENS con puntaje consolidado entero de 1 a 5 en UX_RESPONSE.total_score.

2.2 Requisitos

Tipo	Requisito
Funcional	Registrar usuarios anonimos, jugadores, sesiones, mapas, sectores y eventos de telemetria.
Funcional	Cargar logs reales mediante staging y transformar datos hacia tablas core.
Funcional	Relacionar telemetria con respuestas UX PENS.
No funcional	Mantener integridad referencial mediante PK, FK, UNIQUE y CHECK.
No funcional	Crear indices para consultas por trayectoria, sector, tiempo y jugador.
Etica	Guardar consentimiento, anonimizar participantes y evitar datos personales directos.

2.3 Diagrama ER

El diagrama ER queda sincronizado con el esquema fisico. Incluye source_session_id y source_file en GAME, event_timestamp en TELEMETRY_EVENT y total_score entero en UX_RESPONSE.



2.1 Esquema relacional resumido

Tabla	Llave primaria	Relaciones principales
research_user	user_id	Relacion 1:N con player; relacion 1:N con ux_response.
player	player_id	FK user_id; participa en game mediante game_participant.
episode	episode_id	Relacion 1:N con map.
map	map_id	FK episode_id; relacion 1:N con sector y game.
sector	sector_id	FK map_id; usado por telemetry_event.
game	game_id	FK map_id; contiene source_session_id y source_file.
game_participant	game_id, player_id	Tabla puente Player-Game.
telemetry_event	telemetry_id	FK game_id, player_id, sector_id; UNIQUE(game_id, player_id, tic).
ux_instrument / ux_item / ux_response	instrument_id / item_id / response_id	Instrumento PENS y respuestas enteras 1 a 5.

3. Parte B - Implementacion e ingesta

Archivo	Responsabilidad
01_schema.sql	Crea esquema relacional, PK, FK, UNIQUE y CHECK.
02_staging.sql	Crea staging_real telemetry_raw y etl_error_log.
03_seed_data.sql	Registra 6 usuarios y 6 jugadores.
04_load_ux.sql	Carga PENS con items y respuestas enteras.
05_load_real telemetry_csv.sql	Carga el CSV staging de 6 jugadores.
06_etl_real telemetry.sql	Valida y transforma staging hacia tablas core.
07_indexes.sql	Crea indices principales.
08_validation_queries.sql	Valida cantidades y datos cargados.
09_part_c_analytical_queries.sql	Contiene 8 consultas analíticas.
10_index_evaluation_before_after.sql	Contiene EXPLAIN ANALYZE antes/despues.
11_views_materialized.sql	Crea 2 vistas y 1 vista materializada.
Makefile / recreate_schema_and_load.sh	Mecanismo reproducible para recrear esquema y cargar datos.

3.1 Flujo ETL

Paso	Descripcion
Extraccion	Se toman TXT reales de Chocolate Doom capturados al jugar.
Normalizacion	Los TXT se consolidan en doom_telemetry_unificada.csv con player_nickname y source_file.
Staging	Los campos entran como texto para evitar fallos prematuros por tipos.
Validacion	Se validan requeridos, formatos numericos y existencia de jugador.
Transformacion	Se insertan episodios, mapas, sectores, sesiones, participantes y telemetria.
Core	Los eventos quedan en TELEMETRY_EVENT con timestamp, tic, posicion, momentum, vida, armadura y municion.

3.3 Base completa de 6 jugadores

La base completa se entrega como CSV en el archivo doom_telemetry_staging_6_jugadores.csv, preparado para importar a la tabla staging_real_telemetry_raw. El archivo tiene 5.686 filas de telemetría real y 20 columnas compatibles con el proceso ETL.

Archivo de base	Uso
doom_telemetry_unificada_6_jugadores.csv	CSV limpio con nombres finales de columnas.
doom_telemetry_staging_6_jugadores.csv	CSV preparado para staging con columnas terminadas en _raw. Este es el recomendado para pgAdmin.
resumen_telemetria_6_jugadores.csv	Resumen por jugador para validación y sustentación.
validaciones_telemetria_6_jugadores.sql	Consultas de verificación de conteos, jugadores, sesiones y rangos temporales.

Muestra de la base staging

source_file_raw	player_nickname_raw	source_session_id_raw	session_start_raw	episode_code_raw	map_code_raw	event_timestamp_raw	tic_raw
doom_telemetry_real.csv	player_01	1	2026-06-03 22:07:05	E1	MAP05	2026-06-03 22:07:05	171
doom_telemetry_real.csv	player_01	1	2026-06-03 22:07:05	E1	MAP05	2026-06-03 22:07:07	206
doom_telemetry_real.csv	player_01	1	2026-06-03 22:07:05	E1	MAP05	2026-06-03 22:07:08	241
doom_telemetry_real.csv	player_01	1	2026-06-03 22:07:05	E1	MAP05	2026-06-03 22:07:09	276
doom_telemetry_real.csv	player_01	1	2026-06-03 22:07:05	E1	MAP05	2026-06-03 22:07:10	311

3.4 Evidencia de carga y transformacion

Validacion	Resultado
COUNT(*) en staging_real telemetry_raw	5686
COUNT(*) en telemetry_event despues del ETL	5686
Jugadores con telemetría en tabla final	6
Errores de campos requeridos detectados	0
Total de sesiones reales transformadas	50

3.5 Código clave de carga

```
CREATE TABLE game (  
  game_id SERIAL PRIMARY KEY,  
  map_id INTEGER NOT NULL REFERENCES map(map_id),  
  source_session_id INTEGER UNIQUE NOT NULL,  
  source_file VARCHAR(160),  
  start_time TIMESTAMP NOT NULL,  
  end_time TIMESTAMP,  
  CHECK (end_time IS NULL OR end_time >= start_time)  
);  
  
CREATE TABLE telemetry_event (  
  telemetry_id BIGSERIAL PRIMARY KEY,  
  game_id INTEGER NOT NULL REFERENCES game(game_id) ON DELETE CASCADE,  
  player_id INTEGER NOT NULL REFERENCES player(player_id) ON DELETE CASCADE,  
  sector_id INTEGER NOT NULL REFERENCES sector(sector_id),  
  event_timestamp TIMESTAMP NOT NULL,  
  tic INTEGER NOT NULL CHECK (tic > 0),  
  pos_x NUMERIC(12,4) NOT NULL,  
  pos_y NUMERIC(12,4) NOT NULL,  
  pos_z NUMERIC(12,4) NOT NULL,  
  health INTEGER CHECK (health >= 0),  
  armor INTEGER CHECK (armor >= 0),  
  ammo INTEGER CHECK (ammo >= 0),  
  UNIQUE(game_id, player_id, tic)  
);
```

4. Parte C - Consultas, indices y reporte

4.1 Consultas analiticas implementadas		
#	Consulta	Resultado presentado
Q1	Duracion promedio de sesiones por mapa	Tabla por episodio y mapa.
Q2	Jugadores con mayor proximidad promedio	Pares de jugadores, tics comparables y distancia promedio.
Q3	Trayectorias mas cortas y largas por jugador	Minimo, maximo y promedio de distancia.
Q4	UX de jugadores con duracion superior al promedio	Relacion entre PENS y telemetria.
Q5	Sector mas visitado por episodio y mapa	Hotspots por episodio/mapa.
Q6	Tics donde jugadores estuvieron juntos en un sector	Co-presencia comparable por sector.
Q7	UX promedio para trayectoria mas corta por episodio	Uso de ventana/ranking.
Q8	Distancia total y velocidad promedio por jugador	Trayectoria acumulada con LAG.

4.1 Indices

Se implementaron siete indices. Se incluye una medicion reproducible con EXPLAIN ANALYZE antes y despues de crear

indices. En tablas pequenas PostgreSQL puede decidir usar Seq Scan; aun asi, los indices quedan listos para el crecimiento esperado del dataset.

Indice	Uso
idx_telemetry_game_player_tic	Trayectorias por sesion, jugador y tic
idx_telemetry_event_timestamp	Filtros temporales
idx_telemetry_sector	Hotspots y sector
idx_game_source_session	Busqueda de sesiones
idx_game_source_file	Trazabilidad de captura
idx_sector_map_code	Resolucion de sector
idx_c_te_game_tic_sector_player	Patrones de co-presencia

4.2 Vistas

Objeto	Tipo	Uso
v_player_session_summary	VIEW	Resumen por sesion, jugador, episodio y mapa
v_sector_hotspots	VIEW	Frecuencia de visitas por sector
mv_player_trajectory_distance	MATERIALIZED VIEW	Distancias y velocidad promedio por jugador y sesion

4.3 Espacios para evidencia

Validacion de cantidades y seis jugadores

The screenshot shows a PostgreSQL query editor with the following components:

- Query Editor:** Contains the SQL query:


```
1 SELECT COUNT(*) AS total_staging
2 FROM staging_real_telemetry_raw;
```
- Data Output:** Displays the result of the query in a table format:

	total_staging bigint
1	5686
- Interface Elements:** Includes a toolbar with icons for file operations, a 'No limit' dropdown, and a 'Showing rows: 1 to 1' indicator.

Esta evidencia muestra que el CSV unificado de seis jugadores fue importado correctamente a la tabla staging con 5.686 registros reales.

Resultado de las 8 consultas analíticas

```
Q1: SELECT

    e.episode_code,

    m.map_code,

    COUNT(g.game_id) AS total_sesiones,

    ROUND(AVG(EXTRACT(EPOCH FROM (g.end_time - g.start_time)))::numeric, 2) AS duracion_promedio_segundos

FROM game g

JOIN map m ON m.map_id = g.map_id

JOIN episode e ON e.episode_id = m.episode_id

GROUP BY e.episode_code, m.map_code

ORDER BY duracion_promedio_segundos DESC;
```

Data Output Messages Notifications					
Showing rows: 1 to 10 Page No: 1 of 1					
	episode_code character varying (20)	map_code character varying (20)	total_sesiones bigint	duracion_promedio_segundos numeric	
1	E1	MAP02	1	1194.00	
2	E1	MAP01	4	864.75	
3	E1	MAP04	1	597.00	
4	E3	MAP01	8	568.75	
5	E2	MAP01	2	182.00	
6	E4	MAP01	3	118.00	
7	E2	MAP02	9	61.56	
8	E3	MAP05	6	58.50	
9	E1	MAP05	13	42.08	
10	E4	MAP02	3	23.67	

```

Q2: WITH eventos_comparables AS (

SELECT

    p.nickname,

    e.episode_code,

    m.map_code,

    te.pos_x,

    te.pos_y,

    te.pos_z,

    ROW_NUMBER() OVER (

        PARTITION BY p.player_id, e.episode_code, m.map_code

        ORDER BY te.event_timestamp, te.tic

    ) AS paso_comparable

FROM telemetry_event te

JOIN player p ON p.player_id = te.player_id

JOIN game g ON g.game_id = te.game_id

JOIN map m ON m.map_id = g.map_id

JOIN episode e ON e.episode_id = m.episode_id

),

distancias AS (

SELECT

    a.nickname AS player_a,

    b.nickname AS player_b,

    a.episode_code,

    a.map_code,

    COUNT(*) AS puntos_comparados,

    AVG(SQRT(

```



```

        POWER(a.pos_x - b.pos_x, 2) +

        POWER(a.pos_y - b.pos_y, 2) +

        POWER(a.pos_z - b.pos_z, 2)

    )) AS distancia_promedio

FROM eventos_comparables a

JOIN eventos_comparables b

    ON a.episode_code = b.episode_code

    AND a.map_code = b.map_code

    AND a.paso_comparable = b.paso_comparable

    AND a.nickname < b.nickname

GROUP BY

    a.nickname,

    b.nickname,

    a.episode_code,

    a.map_code

)

SELECT

    player_a,

    player_b,

    episode_code,

    map_code,

    puntos_comparados,

    ROUND(distancia_promedio::numeric, 2) AS distancia_promedio

FROM distancias

ORDER BY distancia_promedio ASC

LIMIT 10;

```

Query

Query History

Scratch Pad X

```
4      e.episode_code,
5      m.map_code,
6      te.pos_x,
7      te.pos_y,
8      te.pos_z,
9      ROW_NUMBER() OVER (
10         PARTITION BY p.player_id, e.episode_code, m.map_code
11         ORDER BY te.event_timestamp, te.tic
```

Data Output

Messages

Notifications

Showing rows: 1 to 10

Page No: 1 of 1

	player_a character varying (80)	player_b character varying (80)	episode_code character varying (20)	map_code character varying (20)	puntos_comparados bigint	distancia_promedio numeric
1	player_04	player_06	E1	MAP05	27	0.00
2	player_03	player_06	E1	MAP05	48	0.00
3	player_04	player_05	E1	MAP05	27	0.00
4	player_03	player_04	E1	MAP05	27	0.00
5	player_03	player_05	E1	MAP05	48	0.00
6	player_02	player_03	E1	MAP05	38	0.00
7	player_02	player_04	E1	MAP05	27	0.00
8	player_02	player_05	E1	MAP05	38	0.00
9	player_02	player_06	E1	MAP05	38	0.00
10	player_05	player_06	E1	MAP05	48	0.00

Q3: WITH puntos AS (

SELECT

te.game_id,

te.player_id,

te.tic,

te.pos_x,

te.pos_y,

te.pos_z,

LAG(te.pos_x) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_x,

LAG(te.pos_y) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_y,

LAG(te.pos_z) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_z

FROM telemetry_event te

),

distancias AS (

SELECT

game_id,

player_id,

SQRT(

POWER(pos_x - prev_x, 2) +

POWER(pos_y - prev_y, 2) +

POWER(pos_z - prev_z, 2)

) AS distancia

FROM puntos

```

WHERE prev_x IS NOT NULL
)
SELECT
    p.nickname,
    ROUND(MIN(s.distancia_total_sesion)::numeric, 2) AS trayectoria_mas_corta,
    ROUND(MAX(s.distancia_total_sesion)::numeric, 2) AS trayectoria_mas_larga
FROM (
    SELECT
        game_id,
        player_id,
        SUM(distancia) AS distancia_total_sesion
    FROM distancias
    GROUP BY game_id, player_id
) s
JOIN player p ON p.player_id = s.player_id
GROUP BY p.nickname
ORDER BY p.nickname;

```

Data Output Messages Notifications				
Showing rows: 1 to 6				
	nickname character varying (80)	trayectoria_mas_corta numeric	trayectoria_mas_larga numeric	
1	player_01	1921.47	112987.63	
2	player_02	4902.90	51236.94	
3	player_03	6575.06	43188.55	
4	player_04	3796.75	33116.94	
5	player_05	280.79	47973.27	
6	player_06	879.37	79198.55	

```

Q4: WITH duracion_jugador AS (
    SELECT
        p.player_id,
        p.nickname,
        SUM(EXTRACT(EPOCH FROM (g.end_time - g.start_time))) AS duracion_total
    FROM game g
    JOIN game_participant gp ON gp.game_id = g.game_id
    JOIN player p ON p.player_id = gp.player_id
    GROUP BY p.player_id, p.nickname
),
promedio AS (

```

```

SELECT AVG(duracion_total) AS promedio_global
FROM duracion_jugador
)
SELECT
    dj.nickname,
    dj.duracion_total,
    ux.total_score
FROM duracion_jugador dj
JOIN player p ON p.player_id = dj.player_id
JOIN research_user ru ON ru.user_id = p.user_id
JOIN ux_response ux ON ux.user_id = ru.user_id
CROSS JOIN promedio pr
WHERE dj.duracion_total > pr.promedio_global
ORDER BY dj.duracion_total DESC;

```

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Show

	nickname character varying (80)	duracion_total numeric	total_score integer	
1	player_03	3296.000000	4	
2	player_01	2919.000000	4	
3	player_05	2865.000000	3	

```

Q5: SELECT
    e.episode_code,
    m.map_code,
    s.sector_code,
    COUNT(*) AS total_visitas
FROM telemetry_event te
JOIN sector s ON s.sector_id = te.sector_id
JOIN map m ON m.map_id = s.map_id
JOIN episode e ON e.episode_id = m.episode_id
GROUP BY e.episode_code, m.map_code, s.sector_code
ORDER BY total_visitas DESC
LIMIT 10;

```

	episode_code character varying (20) 🔒	map_code character varying (20) 🔒	sector_code character varying (40) 🔒	total_visitas bigint 🔒
1	E3	MAP01	(1,-6)	230
2	E3	MAP01	(0,-6)	169
3	E1	MAP05	(0,0)	145
4	E3	MAP01	(1,0)	139
5	E3	MAP01	(0,-1)	125
6	E3	MAP01	(0,0)	105
7	E1	MAP05	(0,2)	97
8	E4	MAP01	(0,0)	95
9	E3	MAP01	(0,-2)	89
10	E1	MAP05	(0,1)	87

Q6: SELECT

```

    pl.nickname AS player_a,

    p2.nickname AS player_b,

    e.episode_code,

    m.map_code,

    s.sector_code,

    COUNT(*) AS coincidencias_sector

FROM telemetry_event tel

JOIN telemetry_event te2

    ON tel.sector_id = te2.sector_id

    AND tel.player_id < te2.player_id

JOIN player p1 ON p1.player_id = tel.player_id

JOIN player p2 ON p2.player_id = te2.player_id

JOIN sector s ON s.sector_id = tel.sector_id

JOIN map m ON m.map_id = s.map_id

JOIN episode e ON e.episode_id = m.episode_id

GROUP BY

    pl.nickname,

    p2.nickname,

    e.episode_code,

    m.map_code,

    s.sector_code

ORDER BY coincidencias_sector DESC

LIMIT 20;

```

	player_a character varying (80)	player_b character varying (80)	episode_code character varying (20)	map_code character varying (20)	sector_code character varying (40)	coincidencias_sector bigint
1	player_02	player_05	E3	MAP01	(1,-6)	10800
2	player_02	player_05	E3	MAP01	(0,-6)	6336
3	player_02	player_05	E3	MAP01	(1,0)	2808
4	player_02	player_05	E3	MAP01	(0,-1)	1976
5	player_02	player_06	E3	MAP01	(0,-1)	1664
6	player_05	player_06	E3	MAP01	(1,-6)	1512
7	player_02	player_06	E3	MAP01	(1,-6)	1400
8	player_01	player_06	E1	MAP05	(0,0)	1380
9	player_01	player_05	E1	MAP05	(0,0)	1380
10	player_01	player_03	E1	MAP05	(0,0)	1380
11	player_02	player_06	E3	MAP01	(0,0)	1353
12	player_05	player_06	E3	MAP01	(0,-1)	1216
13	player_02	player_05	E3	MAP01	(0,0)	1066
14	player_05	player_06	E3	MAP01	(0,-2)	972
15	player_02	player_06	E3	MAP01	(1,0)	918
16	player_05	player_06	E3	MAP01	(1,0)	884
17	player_04	player_05	E3	MAP01	(1,-6)	864
18	player_02	player_04	E3	MAP01	(1,0)	864
19	player_05	player_06	E3	MAP01	(0,0)	858

Q7: WITH puntos AS (

SELECT

e.episode_code,

te.game_id,

te.player_id,

te.tic,

te.pos_x,

te.pos_y,

te.pos_z,

LAG(te.pos_x) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_x,

LAG(te.pos_y) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_y,

LAG(te.pos_z) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_z

FROM telemetry_event te

JOIN game g ON g.game_id = te.game_id

JOIN map m ON m.map_id = g.map_id

JOIN episode e ON e.episode_id = m.episode_id

),

distancias AS (

SELECT

episode_code,

game_id,

player_id,

SQRT(

```

        POWER(pos_x - prev_x, 2) +

        POWER(pos_y - prev_y, 2) +

        POWER(pos_z - prev_z, 2)

    ) AS distancia

FROM puntos

WHERE prev_x IS NOT NULL

),

trayectoria AS (

    SELECT

        episode_code,

        player_id,

        SUM(distancia) AS distancia_total

    FROM distancias

    GROUP BY episode_code, player_id

),

ranking AS (

    SELECT

        *,

        RANK() OVER (PARTITION BY episode_code ORDER BY distancia_total ASC) AS ranking_distancia

    FROM trayectoria

)

SELECT

    r.episode_code,

    p.nickname,

    ROUND(r.distancia_total::numeric, 2) AS distancia_total,

    ROUND(AVG(ux.total_score)::numeric, 2) AS ux_promedio

FROM ranking r

JOIN player p ON p.player_id = r.player_id

JOIN research_user ru ON ru.user_id = p.user_id

JOIN ux_response ux ON ux.user_id = ru.user_id

WHERE r.ranking_distancia = 1

GROUP BY r.episode_code, p.nickname, r.distancia_total

ORDER BY r.episode_code;

```

Data Output Messages Notifications					
☰+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Showing rows: 1 to 4					
	episode_code character varying (20) 🔒	nickname character varying (80) 🔒	distancia_total numeric 🔒	ux_promedio numeric 🔒	
1	E1	player_04	3796.75	5.00	
2	E2	player_06	12784.78	4.00	
3	E3	player_04	33116.94	5.00	
4	E4	player_01	7166.60	4.00	

Q8: WITH puntos AS (

SELECT

te.game_id,

te.player_id,

te.tic,

te.event_timestamp,

te.pos_x,

te.pos_y,

te.pos_z,

LAG(te.pos_x) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_x,

LAG(te.pos_y) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_y,

LAG(te.pos_z) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_z,

LAG(te.event_timestamp) OVER (PARTITION BY te.game_id, te.player_id ORDER BY te.tic) AS prev_time

FROM telemetry_event te

),

distancias AS (

SELECT

game_id,

player_id,

SQRT(

POWER(pos_x - prev_x, 2) +

POWER(pos_y - prev_y, 2) +

POWER(pos_z - prev_z, 2)

) AS distance_units,

EXTRACT(EPOCH FROM (event_timestamp - prev_time)) AS duration_seconds

FROM puntos

WHERE prev_x IS NOT NULL

)

SELECT

p.nickname,


```

ROUND(SUM(d.distance_units)::numeric, 2) AS total_distance,

ROUND(SUM(d.duration_seconds)::numeric, 2) AS total_duration_seconds,

ROUND((SUM(d.distance_units) / NULLIF(SUM(d.duration_seconds), 0))::numeric, 4) AS avg_speed_units_per_second

FROM distancias d

JOIN player p ON p.player_id = d.player_id

GROUP BY p.nickname

ORDER BY total_distance DESC;

```

	nickname character varying (80)	total_distance numeric	total_duration_seconds numeric	avg_speed_units_per_second numeric
1	player_01	365790.34	2873.00	127.3200
2	player_06	203108.18	1676.00	121.1863
3	player_05	105140.56	2853.00	36.8526
4	player_03	82846.43	3289.00	25.1889
5	player_02	56139.84	913.00	61.4894
6	player_04	36913.69	348.00	106.0738

nickname character varying (80)	source_file character varying (160)	episode_code character varying (20)	map_code character varying (20)	telemetry_events bigint	trajectory_distance numeric
player_01	doom_telemetry_completo_23h.txt	E1	MAP02	741	112987.63
player_02	doom_telemetry_player_02_german_...	E2	MAP01	280	48443.74
player_01	doom_telemetry_completo_23h.txt	E1	MAP01	227	35942.13
player_01	doom_telemetry_completo_23h.txt	E1	MAP01	227	35942.13
player_02	doom_telemetry_player_02_german_...	E3	MAP05	102	15734.35
player_01	doom_telemetry_completo_23h.txt	E3	MAP05	102	15734.35
player_02	doom_telemetry_player_02_german_...	E3	MAP05	102	15727.87
player_01	doom_telemetry_completo_23h.txt	E3	MAP05	102	15727.87
player_01	doom_telemetry_completo_23h.txt	E3	MAP05	102	15703.68
player_02	doom_telemetry_player_02_german_...	E3	MAP05	102	15703.68

06_etl_real_teleme... X 11_views_materiali... X Chocolatedoom3/... X 06_etl_real_teleme... X 08_validation_queries.sql X

Chocolatedoom3/postgres@proyectobases

Query Query History Scratch Pad X

```
4 UNION ALL
5 SELECT 'Jugadores con telemetria real', COUNT(DISTINCT player_id) FROM telemetry_event
6 UNION ALL
7 SELECT 'Episodios reales', COUNT(*) FROM episode
8 UNION ALL
9 SELECT 'Mapas reales', COUNT(*) FROM map
10 UNION ALL
11 SELECT 'Sectores reales', COUNT(*) FROM sector
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	primera_muestra timestamp without time zone	ultima_muestra timestamp without time zone	total_muestras bigint
1	2026-06-02 19:30:14	2026-06-04 21:37:00	5686

l_real_teleme... X 11_views_materiali... X Chocolatedoom3/... X 06_etl_real_teleme... X 09_part_c_analytical_queries.sql X

Chocolatedoom3/postgres@proyectobases

Query Query History Scratch Pad X

```
4 JOIN episode e ON e.episode_id = m.episode_id
5 GROUP BY e.episode_code, m.map_code
6 ORDER BY avg_duration_seconds DESC;
7
8 SELECT p1.nickname AS player_a, p2.nickname AS player_b, e.episode_code, m.map_code,
9 FROM telemetry_event tel
10 JOIN telemetry_event te2 ON tel.tic = te2.tic AND tel.player_id < te2.player_id
11 JOIN game g1 ON g1.game_id = tel.game_id
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	nickname character varying (80)	total_distance numeric	total_duration_seconds numeric	avg_speed_units_per_second numeric
1	player_01	365790.34	2873.00	127.3200
2	player_06	203108.18	1676.00	121.1863
3	player_05	105140.56	2853.00	36.8526
4	player_03	82846.43	3289.00	25.1889
5	player_02	56139.84	913.00	61.4894
6	player_04	36913.69	348.00	106.0738

EXPLAIN ANALYZE antes de indices

Chocolatedoom3/postgres@proyectobases

Query

```
1 DROP INDEX IF EXISTS idx_c_te_game_player_tic;
2 DROP INDEX IF EXISTS idx_c_te_game_tic_sector_player;
3 DROP INDEX IF EXISTS idx_c_te_sector;
4 DROP INDEX IF EXISTS idx_c_te_event_timestamp;
5 DROP INDEX IF EXISTS idx_c_gp_player_game;
6 DROP INDEX IF EXISTS idx_c_ux_response_user_instrument;
7 DROP INDEX IF EXISTS idx_c_sector_map_grid;
```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

QUERY PLAN

text

```
1 Sort (cost=345.19..349.04 rows=1542 width=20) (actual time=4.138..4.139 rows=0.00 loops=1)
2   Sort Key: (count(*)) DESC
3   Sort Method: quicksort  Memory: 25kB
4   Buffers: shared hit=92
5   -> HashAggregate (cost=205.72..263.53 rows=1542 width=20) (actual time=4.130..4.131 rows=0.00 loops=1)
6     Group Key: game_id, tic, sector_id
7     Filter: (count(*) > 1)
8     Batches: 1  Memory Usage: 665kB
9     Rows Removed by Filter: 5686
10    Buffers: shared hit=92
11    -> Seq Scan on telemetry_event_te (cost=0.00..148.86 rows=5686 width=12) (actual time=0.019..0.604 rows=5686 loops=1)
Total rows: 14 Query complete 00:00:00.208
```

EXPLAIN ANALYZE despues de indices

Query Query History Scratch Pad X

```

1 EXPLAIN ANALYZE
2 SELECT
3     te.sector_id,
4     COUNT(*) AS total_visitas
5 FROM telemetry_event te
6 GROUP BY te.sector_id
7 ORDER BY total_visitas DESC
8 LIMIT 10;

```

Data Output Messages Notifications

Showing rows: 1 to 17 Page No: 1 of 1

QUERY PLAN

Limit (cost=167.42..167.44 rows=10 width=12) (actual time=0.833..0.835 rows=10.00 loops=1)

Buffers: shared hit=7 read=5

-> Sort (cost=167.42..168.80 rows=551 width=12) (actual time=0.832..0.833 rows=10.00 loops=1)

Sort Key: (count(*)) DESC

Sort Method: top-N heapsort Memory: 25kB

Buffers: shared hit=7 read=5

-> GroupAggregate (cost=0.28..155.51 rows=551 width=12) (actual time=0.103..0.785 rows=551.00 loops=1)

Group Key: sector_id

Buffers: shared hit=4 read=5

-> Index Only Scan using idx_c_te_sector on telemetry_event te (cost=0.28..121.57 rows=5686 width=4) (actual time=0.097..0.483 rows=5686.00 lo...)

Heap Fetches: 0

Total rows: 17 Query complete 00:00:00.052 CRLF Ln 7, Col 28

Resultado de las 2 vistas

11_views_materiali... Chocolatedoom3/... 06_ell_real_teleme... 10_index_evaluatio... 11_views_materialized.sql X

Chocolatedoom3/postgres@proyectobases

No limit

Query Query History Scratch Pad X

```

1 DROP MATERIALIZED VIEW IF EXISTS mv_player_trajectory_distance;
2 DROP VIEW IF EXISTS v_sector_hotspots;
3 DROP VIEW IF EXISTS v_player_session_summary;
4
5 CREATE VIEW v_player_session_summary AS
6 SELECT g.game_id, g.source_session_id, g.source_file, e.episode_code, m.map_code, p.p
7 FROM telemetry_event te

```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

game_id	source_session_id	source_file	episode_code	map_code	player_id	nickname	tele
8	26	doom_telemetry_real.csv	E1	MAP02	1	player_01	
39	51	doom_telemetry_player_04_raw...	E1	MAP04	6	player_06	
43	28	telemetry_jugador1.txt	E3	MAP01	2	player_02	
46	40	telemetry_jugador4.txt	E3	MAP01	5	player_05	
50	46	doom_telemetry_player_04_raw...	E3	MAP01	6	player_06	
27	32	telemetry_jugador2.txt	E2	MAP01	3	player_03	
45	38	telemetry_jugador4.txt	E3	MAP01	5	player_05	
1	4	doom_telemetry_real.csv	E1	MAP01	1	player_01	
3	12	doom_telemetry_real.csv	E1	MAP01	1	player_01	
44	34	telemetry_jugador3.txt	E3	MAP01	4	player_04	

Query

Query History

Scratch Pad

1

SELECT *

2

FROM v_player_session_summary;

3

LIMIT 10;

Data Output

Messages

Notifications

Showing rows: 1 to 10

Page No: 1 of 1

	varying (80)	first_event_time timestamp without time zone	last_event_time timestamp without time zone	telemetry_events bigint	tica_recorded bigint	avg_health numeric	avg_armor numeric	avg_ammo numeric
1		2026-06-03 22:07:05	2026-06-03 22:07:53	48	48	52.42	52.73	44.75
2		2026-06-03 22:08:03	2026-06-03 22:09:08	66	66	84.06	82.08	6.83
3		2026-06-03 22:09:19	2026-06-03 22:09:31	13	13	93.85	0.00	50.00
4		2026-06-03 22:09:36	2026-06-03 22:14:09	227	227	99.03	63.93	40.61
5		2026-06-02 19:30:14	2026-06-02 19:30:37	23	23	78.61	55.04	47.61
6		2026-06-02 19:30:44	2026-06-02 19:31:33	38	38	100.21	37.45	48.05
7		2026-06-03 21:51:46	2026-06-03 21:52:34	48	48	52.42	52.73	44.75
8		2026-06-03 21:52:44	2026-06-03 21:53:14	34	34	95.62	88.68	13.26
9		2026-06-03 22:07:05	2026-06-03 22:07:53	48	48	52.42	52.73	44.75
10		2026-06-03 22:08:03	2026-06-03 22:09:08	66	66	75.42	82.08	6.83

Resultado de la vista materializada

Query		Query History		Scratch Pad	
1		SELECT matviewname			
2		FROM pg_matviews			
3		WHERE schemaname = 'public'			
4		ORDER BY matviewname;			
Data Output		Messages		Notifications	
Showing rows: 1 to 1		Page No: 1		of 1	
matviewname		name			
1		mv_player_trajectory_distan...			

Query

Query History

Scratch Pad X

1

2

3

4

5

6

7

8

SELECT

nickname,

source_file,

episode_code,

map_code,

telemetry_events,

trajectory_distance,

avg_speed_units_per_second

Data Output

Messages

Notifications

<

Importación de CSV de 6 jugadores a staging

Query
Query History
Scratch Pad X

```

1: SELECT p.nickname, COUNT(*) AS total_eventos
2: FROM telemetry_event te
3: JOIN player p ON p.player_id = te.player_id
4: GROUP BY p.nickname
5: ORDER BY p.nickname;

```

Data Output
Messages
Notifications

Showing rows: 1 to 6
Page No: 1 of 1

	nickname character varying (80)	totalEventos bigint
1	player_01	2326
2	player_02	509
3	player_03	492
4	player_04	292
5	player_05	694
6	player_06	1373

5. Sincronizacion ER - esquema - diccionario

Elemento revisado	Estado	Observacion
source_session_id	Corregido	Existe en GAME, aparece en ER y diccionario.
source_file	Corregido	Permite identificar la captura original.
event_timestamp	Corregido	Existe en TELEMETRY_EVENT, consultas y diccionario.
player_nickname_raw	Corregido	Existe en staging para asignar cada captura al jugador correcto.
PENS total_score	Corregido	Se almacena como INTEGER entre 1 y 5.
Datos reales	Corregido	Se consolidan seis jugadores reales.

6. Diccionario de datos actualizado

Tabla	Campo	Tipo	Llave / restriccion	Descripcion
RESEARCH_USER	user_id	SERIAL	PK	Identificador interno del participante.
RESEARCH_USER	anonymous_code	VARCHAR(50)	UNIQUE NOT NULL	Codigo anonimo del usuario.
RESEARCH_USER	age	INTEGER	CHECK age >= 18	Edad minima por etica del estudio.
PLAYER	player_id	SERIAL	PK	Identificador del

				jugador.
PLAYER	user_id	INTEGER	FK	Usuario anonimo asociado.
PLAYER	nickname	VARCHAR(80)	UNIQUE	Alias de juego.
EPISODE	episode_id	SERIAL	PK	Identificador del episodio.
EPISODE	episode_code	VARCHAR(20)	UNIQUE	Codigo E1, E2, E3, E4.
MAP	map_id	SERIAL	PK	Identificador del mapa.
MAP	episode_id	INTEGER	FK	Episodio al que pertenece.
SECTOR	sector_id	SERIAL	PK	Identificador del sector.
SECTOR	grid_x, grid_y	INTEGER	NOT NULL	Coordenadas discretas del sector.
GAME	game_id	SERIAL	PK	Identificador de sesion.
GAME	source_session_id	INTEGER	UNIQUE NOT NULL	Id de sesion desde cada bloque When.
GAME	source_file	VARCHAR(160)	-	Archivo fuente de la sesion.
TELEMETRY_EVENT	telemetry_id	BIGSERIAL	PK	Identificador del evento.
TELEMETRY_EVENT	game_id, player_id, sector_id	INTEGER	FK	Relacion con sesion, jugador y sector.
TELEMETRY_EVENT	event_timestamp	TIMESTAMP	NOT NULL	Fecha y hora del evento.
TELEMETRY_EVENT	tic	INTEGER	UNIQUE game-player-tic	Indice temporal del motor.
TELEMETRY_EVENT	pos_x, pos_y, pos_z	NUMERIC(12,4)	NOT NULL	Coordenadas del jugador.
TELEMETRY_EVENT	health, armor, ammo	INTEGER	CHECK >= 0	Estado de combate.
UX_RESPONSE	total_score	INTEGER	CHECK 1-5	Puntaje PENS entero.

7. Conclusiones

La base queda unificada con telemetria real de seis jugadores y 5686 eventos. El diseno ER, el DDL y el diccionario fueron sincronizados para incluir event_timestamp, source_session_id y source_file. La entrega implementa 8 consultas analiticas, 7 indices, 2 vistas, 1 vista materializada y mecanismo reproducible mediante Makefile y script shell.

El volumen todavia no llega a 20.000 eventos, pero se prioriza no inventar ni duplicar datos. El modelo y el ETL quedan preparados para seguir agregando sesiones reales sin modificar la estructura.